

ProjWT — System Architecture

Project: ProjWT (personal portfolio project — private) **Owner:** Hamza Khan **Last updated:** 2026-10-01
Status: Phase 1–2 complete (BLE + biometrics); gesture & integration layers in design

PRIVATE / PORTFOLIO. Not for distribution. See BUILD_LOG_PWT.md for the step-by-step reverse-engineering record this architecture is built on.

0. Purpose & guiding principles

Build a personal system that reads a WHOOP 4.0 band over BLE and uses it for a biometric dashboard, gesture control, supervised vehicle actions, and home/IoT automation. Eventual target: C++ / Unreal Engine 5; Phase-1 tooling is Python.

Non-negotiable principles

- 1 **Safety first, fail-closed.** Anything that moves a physical object (a car) defaults to **stopped**. Motion requires a continuous, affirmative, supervised signal; the absence of that signal = stop.
- 1 **Line-of-sight, supervised only.** No no-line-of-sight summon, no GPS/location spoofing, no bypassing a vehicle's own safety systems. We only ever invoke **documented** vehicle commands.
- 1 **The band is a sensor, not an authority.** It feeds signals; a separate safety state machine decides whether anything happens.
- 1 **Honest capability boundaries.** We design around what the hardware actually exposes (see §2), not what we wish it exposed.

1. Hard constraint discovered in Phase 2 (shapes everything below)

Reverse-engineering established what the band streams **live over BLE** vs. not:

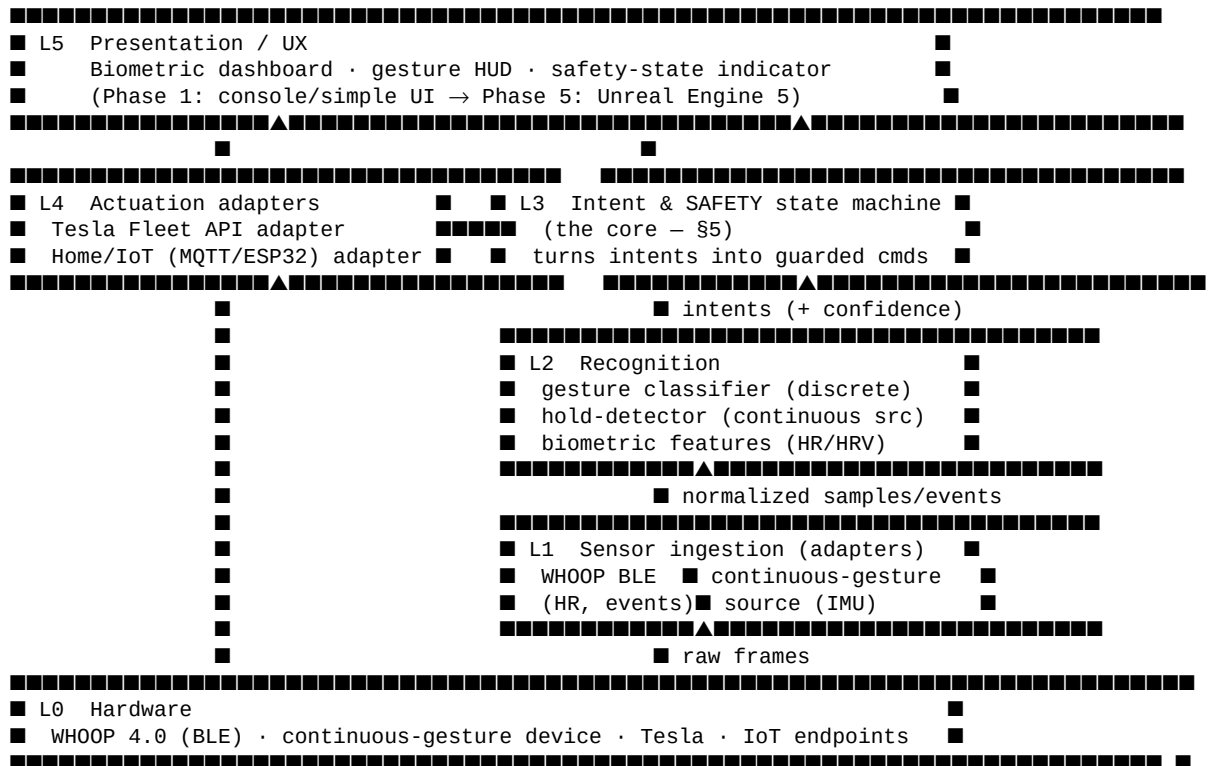
Signal	Live over BLE?	Rate	Use
Heart rate + RR intervals (HRV)	■ yes (type 0x28)	~1 Hz	Biometrics, context
Battery / device info	■ yes	on request	Status
Discrete events (tap / motion / status)	■ yes (type 0x30, EVT(04))	event-driven	Discrete gestures
Raw accelerometer X/Y/Z	■ no (swept opcodes 0x01–0x2f)	—	history-only

The accelerometer is **recorded to flash and only retrievable as historical data** (0x16 dump, calendar-timestamped), not a live feed. **Therefore the band alone cannot provide a smooth,**

continuous wrist-orientation signal — which is exactly what a safe analog "active-hold" control would want.

Consequence: the continuous "hold" signal for vehicle control must come from a **second continuous source**. The WHOOP contributes (a) biometrics and (b) discrete gestures. See §4 for the options and the chosen design.

2. Layered architecture



Why layered: each layer has one job and a stable interface, so Phase-1 Python can be swapped for Phase-5 C++/UE5 one layer at a time, and the safety state machine (L3) is isolated and independently testable.

3. Module map (interfaces, not implementations)

- **L1 WHOOP adapter** — owns the BLE session (pair, keepalive, reconnect; all solved in Phase 2). Emits normalized events: `Biometric{ts, hr, rr[]}` and `BandEvent{ts, report_id, fields}`. Phase-1 impl = `whoop_protocol_PWT.py` + the `ble_*_PWT.py` tools.
- **L1 continuous-gesture adapter** — emits `Pose{ts, ax, ay, az[, gyro]}` at ≥ 25 Hz from the chosen continuous source (§4). Interchangeable behind this type.
- **L2 recognition**
- **GestureClassifier** — discrete gestures from `BandEvent` (tap, double-tap,

maybe flick). Output: `Gesture{kind, confidence, ts}`.

- **HoldDetector** — from Pose: is the supervised hold-pose currently held?

Output: `Hold{active: bool, quality, ts}` at the Pose rate.

- **BiometricFeatures** — rolling HR, HRV, trend from Biometric.

- **L3 Intent & Safety state machine** — the only thing allowed to authorize motion. Consumes Gesture, Hold, Biometric; emits guarded Commands. §5.

- **L4 actuation adapters** — translate Command → concrete API calls, and report acks/failures back. `TeslaFleetAdapter`, `IotAdapter`. §6.

- **L5 presentation** — read-only view of everything + big safety-state banner.

Data contracts are plain structs/JSON so the Python→C++ port is mechanical.

4. The continuous-hold signal (design decision)

A safe active-hold needs a **continuous, high-rate, affirmative** signal. The band can't give it live. Options considered:

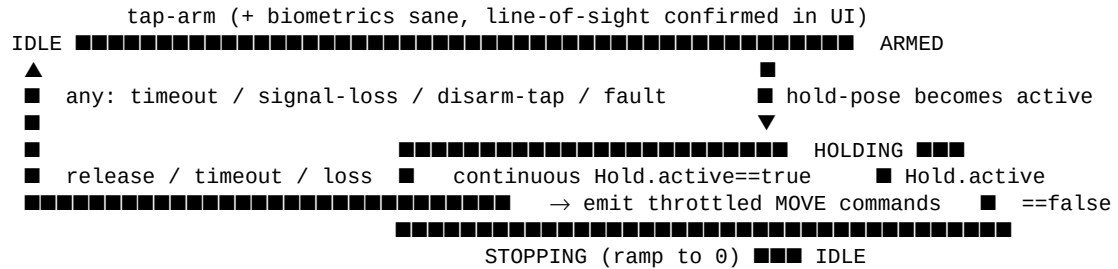
Option	Continuous?	Pros	Cons
A. Phone IMU app (phone in hand, held pose streams over Wi-Fi/USB)	■ ~50 Hz	Already owned; rich orientation; easy dead-man's-switch	Need a small phone app/bridge
B. Second IMU wearable (e.g. ESP32+MPU6050 on wrist)	■ high	Cheap; wrist-mounted like the vision	Build/flash hardware
C. Held physical trigger (BLE/USB button, dead-man's-switch)	■ binary	Simplest, most certifiable fail-safe	Not a "gesture"
D. Band taps only (no continuous source)	■	No extra hardware	Unsafe for motion — can't detect release reliably

Chosen for the vehicle phase: Option A (phone IMU) as the continuous hold source, with Option C (physical trigger) as an optional hardware dead-man's-switch in series. The WHOOP provides biometrics + discrete "arm/disarm" taps and the HUD. Option D is explicitly rejected for anything that moves the car.

This keeps the WHOOP central to the **experience** (it's the wearable you interact with and whose biometrics gate actions) while sourcing the safety-critical continuous signal from a device that actually streams it live.

5. Intent & Safety state machine (the core)

A single, well-tested state machine authorizes all physical motion. Default state is IDLE (stopped). Transitions require affirmative input; loss of input always falls back toward IDLE.



Guards & fail-safes (all must hold to stay in HOLDING):

- `Hold.active == true` refreshed within **200 ms** (watchdog). Any gap → STOPPING.
- Continuous-source heartbeat present (link alive). Loss → STOPPING.
- Biometrics sane (e.g. user conscious/engaged) → optional gate from HR.
- An explicit **disarm tap** or UI release → STOPPING immediately.
- Command rate-limited and **velocity-capped**; commands carry a short TTL so a stale command can't be replayed by the actuator.
- STOPPING ramps output to zero then returns to IDLE; it cannot re-enter HOLDING without a fresh arm+hold cycle.

Testability: L3 is pure logic over input events → it runs in unit tests with synthetic event streams (no hardware), including every fail-safe path. This is the piece we test hardest before any actuator is connected.

6. Actuation adapters — and an honest Tesla note

Tesla Fleet API reality (verify against current Tesla developer docs before building): the Fleet API exposes **discrete, documented commands** (wake, lock/ unlock, climate, charging, honk, flash, vent, trunk/frunk, sentry, etc.). It is **not** a real-time driving interface — you cannot stream steering/throttle to a Tesla over the Fleet API, and we would not bypass the car's own safety if it were possible. Continuous "summon"-style movement in Tesla's own app runs **on the phone with its own hold-to-move + line-of-sight enforcement**.

Therefore the vehicle feature is scoped as:

- **Discrete supervised commands** via the Fleet API (e.g. honk/flash to locate, climate pre-conditioning, lock/unlock, charge-port) — gated by the L3 state machine and a WHOOP tap + confirmation.
- **Any continuous movement** is delegated to **Tesla's native Smart Summon**,

which keeps the human in the loop by design; ProjWT does not re-implement or circumvent it. Our "active-hold" prototype (phone IMU + state machine) is validated against a **simulator / a harmless actuator first** (e.g. a desk robot, or honk/flash), never as a way to drive a real car outside Tesla's own controls.

IotAdapter (home/garage via MQTT/ESP32) is the low-risk first actuator to prove the whole pipeline end-to-end (tap-arm → hold → "open/close/toggle") safely.

7. Technology & phase mapping

Layer	Phase 1 (now, Python)	Phase 5 (target, C++/UE5)
L1 WHOOP	bleak + whoop_protocol_PWT.py	WinRT/BlueZ BLE in a C++ worker thread
L1 continuous src	phone IMU bridge (UDP/JSON)	same, or native sensor
L2 recognition	NumPy / simple DSP + thresholds/DTW	C++ DSP; optional small ML model
L3 safety SM	pure-Python module + unit tests	ported C++ state machine (same tests)
L4 Tesla/IoT	requests/MQTT client	C++ HTTP/MQTT client
L5 UI	console / lightweight web	Unreal Engine 5 HUD

Interfaces (the structs in §3) stay identical across phases, so porting is layer-by-layer, not a rewrite.

8. Build order (de-risked)

- 1 **L1/L2 on the bench (current):** finish decoding live events → discrete gesture classifier; HR/HRV features. *(No actuators.)*
 - 1 **L3 safety state machine + full unit tests** against synthetic events. Prove every fail-safe (timeout, loss, disarm) with no hardware.
 - 1 **L4 IoT first:** wire L3 to a harmless home actuator (garage/LED). Validate the tap-arm → hold → release pipeline end-to-end safely.
 - 1 **Continuous-hold source (phone IMU)** integrated as the HoldDetector input.
 - 2 **Tesla adapter:** discrete documented commands only, behind L3 + confirmation; continuous movement stays with Tesla Smart Summon.
 - 1 **UE5 port**, layer by layer, once behavior is proven in Python.
-

9. Open items feeding this design

- Decode the live EVT(04) events → which report-id = which gesture (in progress, ble_events_PWT.py + labeled captures).
- Confirm current Tesla Fleet API command set & auth flow against official docs.
- Choose/stand up the continuous-hold source (phone IMU bridge spec).

ProjWT architecture — private portfolio project. Keep in sync with BUILD_LOG_PWT.md.